

Strategy pattern

Un pattern de comportement

Préparé par MAA S.Yahiouche

Strategy Pourquoi?

- Je dispose de plusieurs manières de traiter un problème, et ...
- Je dois choisir la manière (la stratégie) au moment de l'exécution...
- C'est le polymorphisme dynamique pratiqué au niveau classes (classiquement on parle de polymorphisme dynamique au niveau des méthodes – redéfinition)

Strategy exemple explicatif

- Supposant avoir une classe Animal, avec des sous classes Chien, Canari, Aigle, pigeon, chauve souris, lapins...
- On voudrait spécifier le comportement voler pour quelques sous classes: fly()
- Ce n'est pas bon de l'inclure dans la super classe car le chien et le lapin ne peuvent pas voler

N'implémentez pas pour une classe!

```
public class Animal {  
  
    private String name;  
    private String sound;  
  
    public void setSound(String newSound){ sound = newSound; }  
  
    public String getSound(){ return sound; }  
    //autres méthodes  
    /.....
```

/* mauvais!

Ce n'est pas bien de rajouter des méthodes dans la super classe qui ne seront pas utilisés par toutes les sous classes:

```
    public void fly(){  
        System.out.println("I'm flying");  
    }  
    */ }
```

Implémentez pour une interface!

```
public interface Flys {  
    String fly();  
}
```

```
public class Animal {  
    //les attributs  
    //.....  
    //les méthodes  
    //.....  
  
    /// Nous allons utiliser une variable d'instance qui est une  
    sous classe de l'interface Flys, c'est la composition }  
  
public Flys flyingType;
```

```
// la classe Animal n'est pas concerné par ce que fait la  
//variable flyingType; elle sait seulement que le  
//comportement sera disponible pour ses sous-classes:
```

```
public String tryToFly() {  
    return flyingType.fly();  
}
```

//si l'on veut changer dynamiquement le type de vol,
flyingType, on peut utiliser la méthode suivante:

```
public void setFlyingAbility(Flys  
newFlyType){  
    flyingType = newFlyType;}
```

Les différents types de comportements(de stratégies)

// Class used if the Animal
can fly

class ItFlys implements
Flys{

public String fly() {
 return "Flying High";

}}

//Class used if the Animal
can't fly

class CantFly implements
Flys{

public String fly() {
 return "I can't fly";
 }}

Les sous classes de Animal

```
public class Dog extends Animal{
    public void digHole(){
        System.out.println("Dug a hole");}
    public Dog(){
        super();
        setSound("Bark");

        // We set the Flys interface
        polymorphically
        // This sets the behavior as a non-flying
        Animal

        flyingType = new CantFly();
    }
}
```

```
public class Aigle extends Animal{

    // The constructor initializes all objects

    public Aigle(){
        super();
        setSound("Tweet");
        // We set the Flys interface
        polymorphically
        // This sets the behavior as a flying
        Animal

        flyingType = new ItFlies();
    }
}
```

Tester les stratégies

```
public class AnimalPlay{

    public static void main(String[] args){

        Animal sparky = new Dog();
        Animal tweety = new Bird();

        System.out.println("Dog: " + sparky.tryToFly());

        System.out.println( " Aigle: " + tweety.tryToFly());

        // l'instruction suivante permet le changement dynamique de flyingType:

        sparky.setFlyingAbility(new ItFlies());

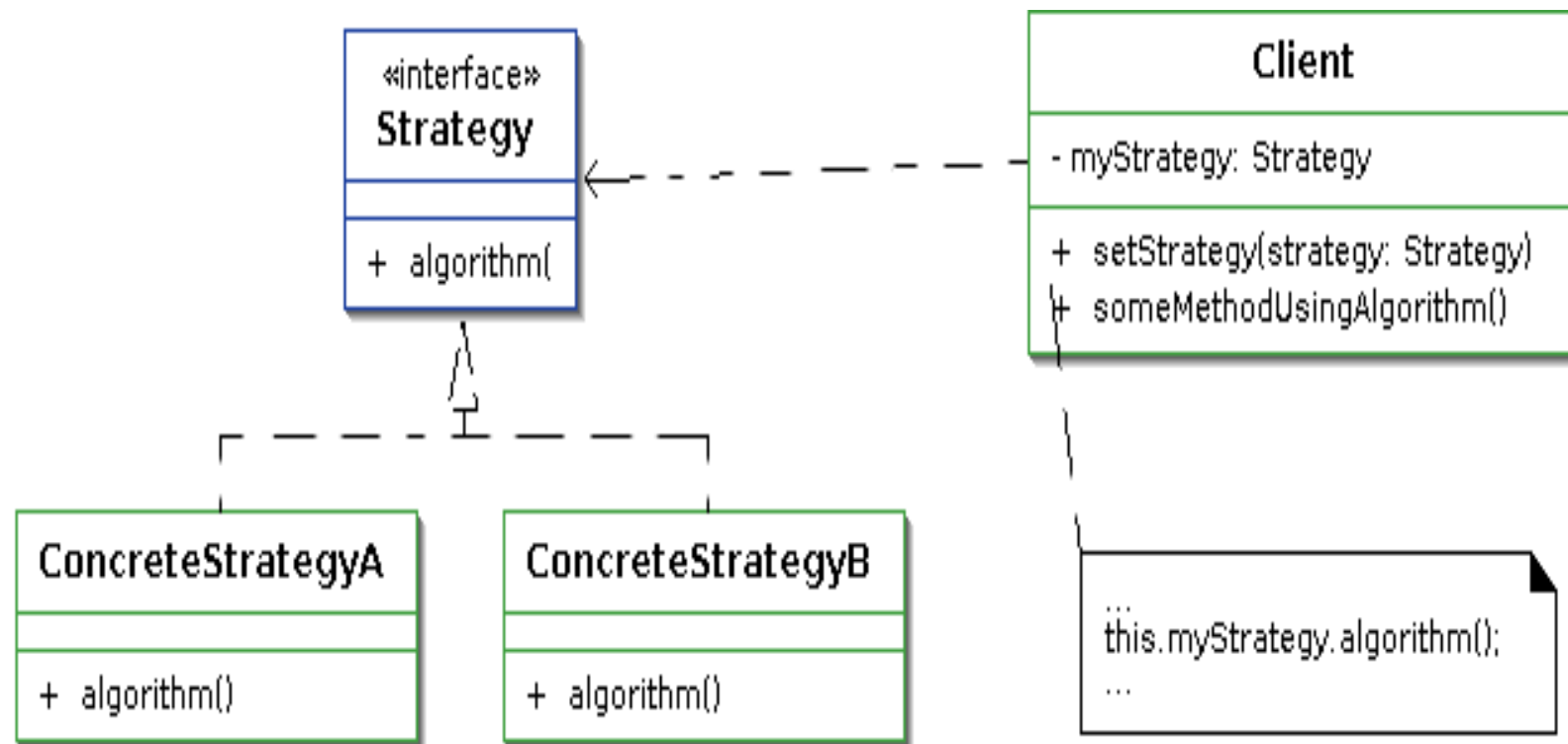
        System.out.println("Dog: " + sparky.tryToFly());

    }
```

Objectif du Strategy pattern

- Définir une famille d'algorithmes, encapsuler chacun d'eux (en tant qu'interfaces) et les rendre interchangeables
- Pouvoir permuter dynamiquement les algorithmes utilisés dans une application au moment de l'exécution

Strategy: Diagramme uml



Strategy dans Java: API Layout Manager

- Dans les api awt/swing, un objet *container* gère la disposition des différents objets graphiques qu'il contient grâce à un *LayoutManager*.
- Le *LayoutManager* modélise les stratégies de placement du *Container*
- public interface `LayoutManager`, ses implémentations:
`java.awt.FlowLayout`, `java.awt.GridLayout`,
`java.awt.BorderLayout`,..(ce sont les stratégies)
- dans `java.awt.Container`, la méthode:
`public void setLayout( LayoutManager mgr)` permet de fixer/
changer la stratégie de placement adoptée par le *Container*

Exemple 2

- On voudrais formater un texte soit en Lettre majuscules soit en miniscules; un éditeur de texte doit choisir le format approprié selon la demande de l'utilisateur.
-

L'interface Strategy et ses classes concretes

```
public interface TextFormatter {  
    public void format(String text);  
}  
public class CapTextFormatter implements TextFormatter{  
  
    public void format(String text) {  
  
        System.out.println("[CapTextFormatter]: "+text.toUpperCase());  
    }  
}  
public class LowerTextFormatter implements TextFormatter{  
  
    public void format(String text) {  
  
        System.out.println("[LowerTextFormatter]: "+text.toLowerCase());  
    }  
}
```

classe utilisant les stratégies :

```
public class TextEditor {  
    private final TextFormatter textFormatter;  
  
    public TextEditor(TextFormatter textFormatter)  
        { this.textFormatter = textFormatter; }  
  
    public void publishText(String text)  
        { textFormatter.format(text); }  
}
```

tester les stratégies

```
public class TestStrategyPattern {  
  
    public static void main(String[] args) {  
        TextFormatter formatter = new CapTextFormatter();  
  
        TextEditor editor = new TextEditor(formatter);  
        editor.publishText("Testing text in caps formatter");  
  
        formatter = new LowerTextFormatter();  
        editor = new TextEditor(formatter);  
        editor.publishText("Testing text in lower formatter");  
    }  
}
```